



哈爾濱工業大學 (深圳)
HARBIN INSTITUTE OF TECHNOLOGY

实验报告

课程名称: 机器学习

实验名称: 实验二: 基于 PyTorch 的手写数字识别

实验性质: 设计型

实验学时: 12 地点: T2102

学生学号: 2024311278

学生姓名: 刘楷

实验与创新实践教育中心制

2026 年 3 月

一、 实验目的

- 1、理解 BP 神经网络的基本原理（前向传播、反向传播、梯度下降）。
- 2、掌握使用 PyTorch 搭建全连接 BP 网络，完成 MNIST 手写数字分类。
- 3、掌握深度学习完整流程：数据加载 → 预处理 → 模型构建 → 训练 → 评估。
- 4、学会调整超参数（batch size、网络深度、神经元数量、激活函数、Dropout），分析其对模型性能的影响。
- 5、能够绘制训练曲线、完成对比实验并撰写规范的实验报告。

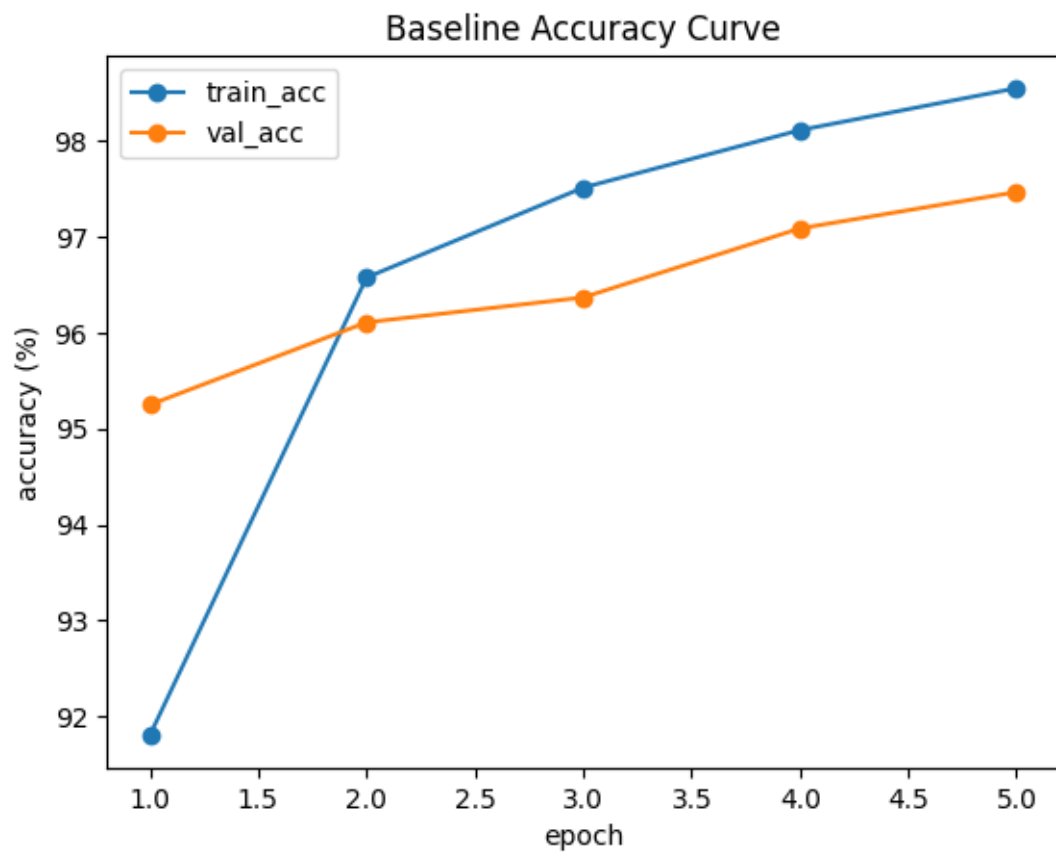
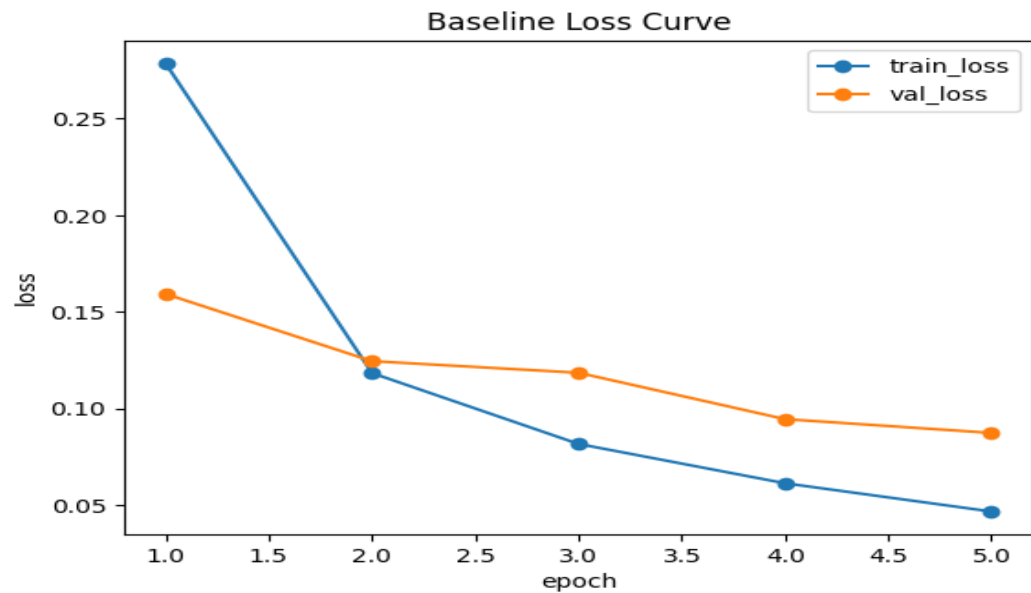
二、 实验环境与工具

（以下内容请根据你的实际情况修改）

- 操作系统：Arch-linux
- Python 版本：Python-3.12.12
- 深度学习框架：PyTorch
- 主要库：torch, torchvision, matplotlib, numpy, scikit-learn
- 开发环境：VS Code
- 硬件：GPU 4060

三、 实验内容与步骤

1. 任务一：环境搭建与基线运行



截图：最终 test 运行结果（测试集的 loss 和 4 个评价指标）。

```
Baseline test loss: 0.0778
Baseline test accuracy: 97.63%
Baseline test precision (macro): 0.9762
Baseline test recall (macro): 0.9761
Baseline test f1-score (macro): 0.9761
```

2. 任务二：模型调参与对比实验

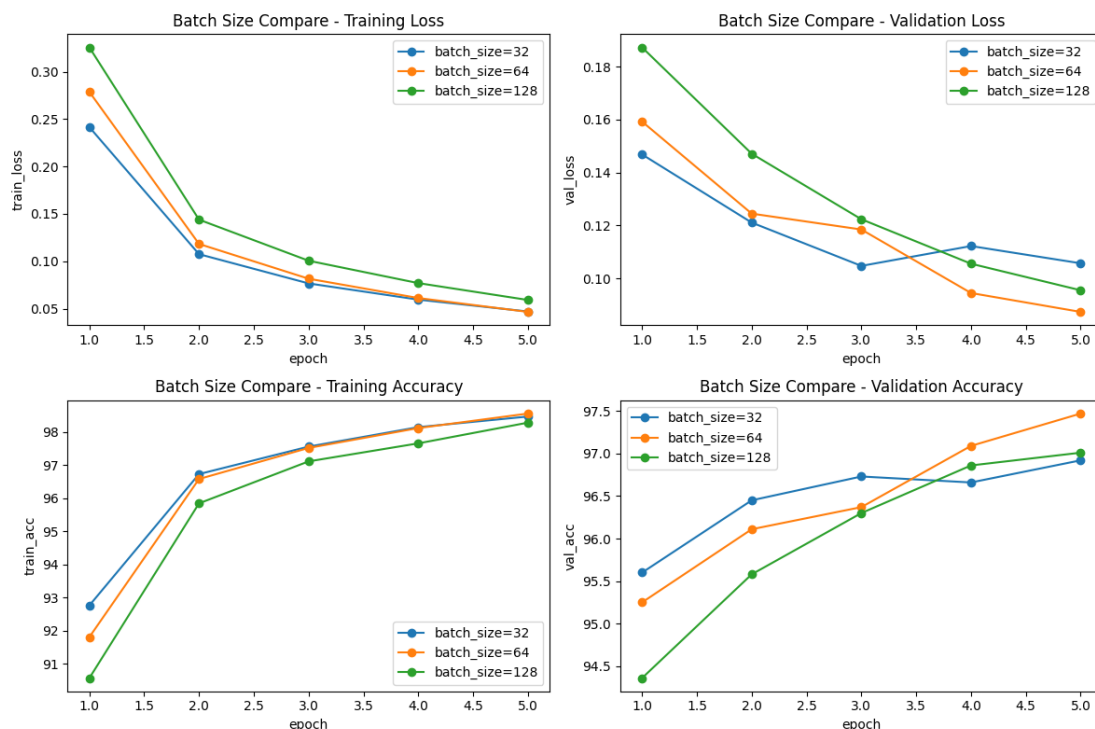
基线模型设置

超参数	内容
网络结构	Flatten → Linear(784, 128) → ReLU → Linear(128, 10)
批量大小 (batch_size)	64
优化器	Adam(lr=0.001)
损失函数	CrossEntropyLoss()
训练轮数 (epoch)	5
数据划分	train=50000, val=10000, test=10000
数据预处理	ToTensor() + Normalize((0.1307,), (0.3081,))

控制变量原则：每次只改变一个超参数，其他保持与基线一致。

(1) 不同批量大小

设置不同 batch_size 大小，训练评估模型。截图训练集和验证集的 loss 曲线和 accuracy 曲线。



比较不同批量大小下 `train_loss` | `train_acc` | `val_loss` | `val_acc` | `time(s)`等数据的区别，简要分析哪组效果最好，可能原因是什么。

Train loss 和 Train Acc

`batch_size=32` 和 `batch_size=64` 的收敛速度最快，两者的训练损失下降和准确率上升的曲线非常接近，且都在第五个 `epoch` 时候训练准确率最高

Val loss 和 Val Acc

`Batch_size=32` 的初期表现很好，到了第三个 `epoch` 的时候，验证损失出现上升，准确率也有了波动，表明模型出现了过拟合现象。

`Batch_size=64` 表现最佳，在五个 `epoch` 中稳步下降，最终到达最低值

`Batch_size=128` 前期较差，收敛也较慢，但是没有出现明显过拟合，验证准确率已经超过了 `batch_size=32` 的组别

Times(s)

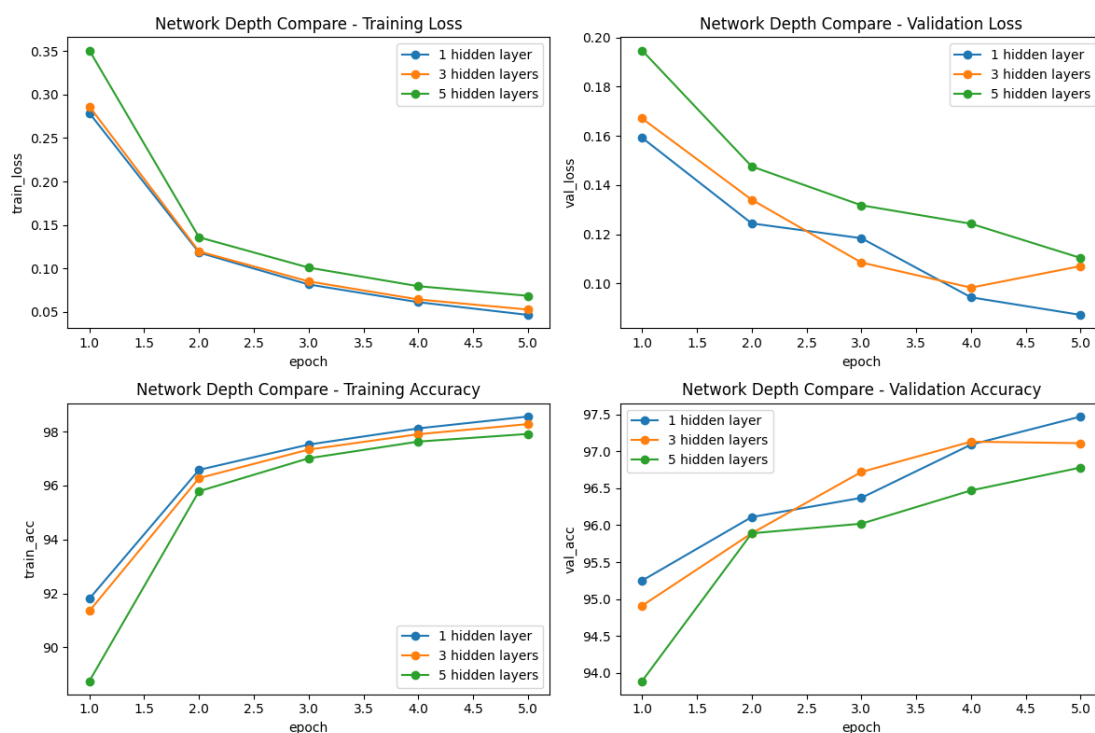
`Batch_size` 训练耗时 $123 < 64 < 32$

Batch_size=64 这组效果最好

可能原因是，batch_size 过小会包含较大噪声，过大会缺少随机噪声点的正则化效果

(2) 不同网络深度（隐藏层层数）

设置不同网络深度（隐藏层层数），训练评估模型。截图训练集和验证集的 loss 曲线和 accuracy 曲线。



比较不同网络深度下 train_loss | train_acc | val_loss | val_acc | time(s) 等数据的区别，简要分析哪组效果最好，可能原因是什么。

Train Loss 和 Train Acc

一个隐藏层和三个 隐藏层 表现非常接近，收敛迅速。其中 一个隐藏层性能上略微胜出，在第 5 个 Epoch 时取得了最低的训练损失和最高的训练准确率。

5 个隐藏层的收敛速度最慢，在所有的 Epoch 中，它的训练损失始终最高，训练准确率始终最低。

Val Loss 和 Val Acc

一个隐藏层整体表现最好，其验证损失持续下降，在第 5 个 Epoch 达到最低

(低于 0.09); 验证准确率也呈上升趋势, 最终达到最高。

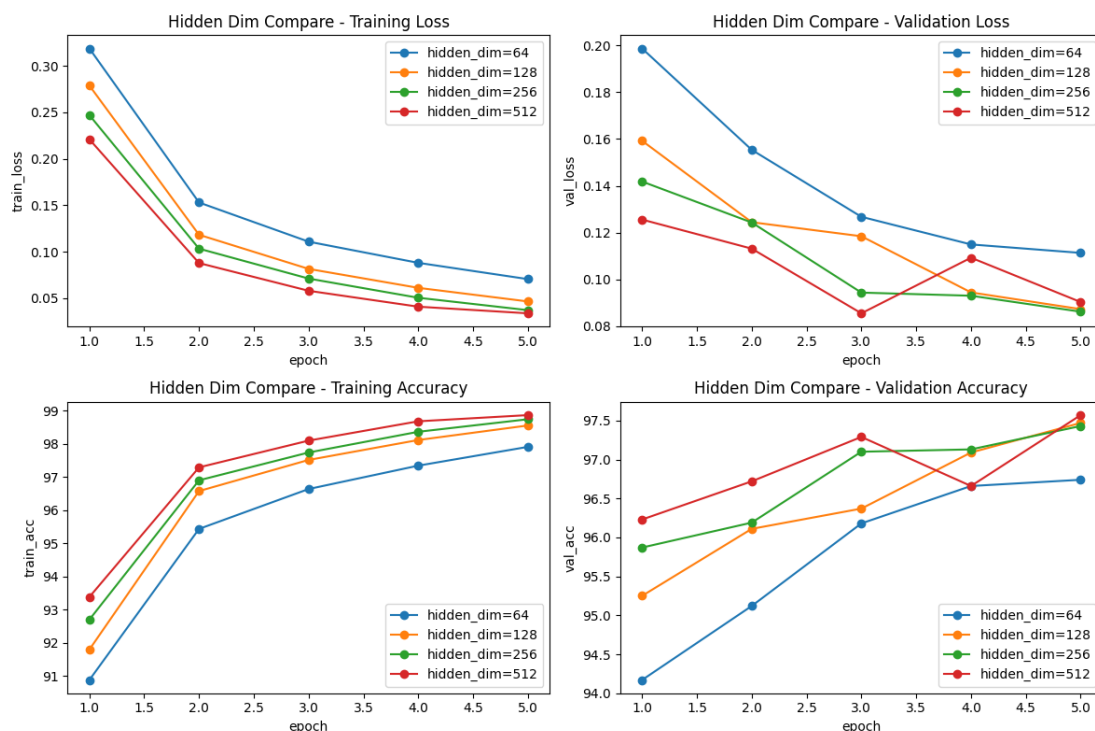
三个隐藏层在前 4 个 Epoch 表现不错, 但在第 5 个 Epoch 时, 其验证损失出现了上升。

五个隐藏层的验证指标全程倒数第一, 尽管曲线在稳步变好, 但由于整体收敛较慢, 未能在 5 个 Epoch 内达到其他浅层网络的水平。

可能原因: 网络过深会导致过拟合, 同时深层网络还会出现退化问题, 和拟合困难

(3) 不同神经元数量 (网络宽度)

设置不同神经元数量, 训练评估模型。截图训练集和验证集的 loss 曲线和 accuracy 曲线。



比较不同神经元数量下 `train_loss` | `train_acc` | `val_loss` | `val_acc` | `time(s)` 等数据的区别, 简要分析哪组效果最好, 可能原因是什么。

Train Loss 和 Train Acc

神经元数量越多, 模型在训练集上的拟合能力越强, hidden_dim=512 的训练损失下降最快、最终值最低, 同时训练准确率上升最快、最终

值最高。hidden_dim=64 的拟合能力相对最弱，在所有 Epoch 中训练损失最高，训练准确率最低。

Val Loss 和 Val Acc

hidden_dim=64: 表现稳定，没有出现过拟合，但是整体验证准确率较低

hidden_dim=128: 验证损失在第 5 个 Epoch 时出现了上升的趋势，准确率增速放缓

hidden_dim=256: 表现出了极佳的稳定性和泛化能力。损失在五个 epoch 训练中稳步下降。

hidden_dim=512: 在第四个 epoch 出现上升趋势，出现过拟合特征。

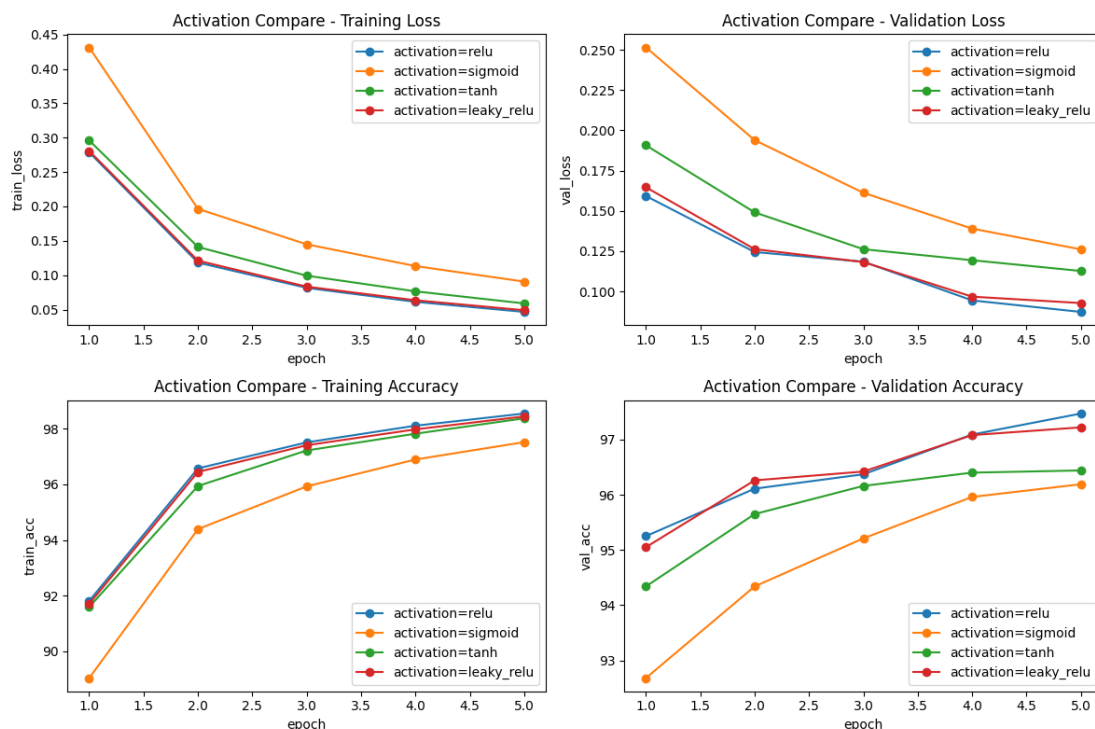
Time

单个 Epoch 耗时排序: 64<128<256<512

可能原因: 神经元适中可以映射当前数据集的特征空间，不会因为太小而损失精度，也不会因为太大而去拟合噪声

(4) 不同激活函数

设置不同激活函数，训练评估模型。截图训练集和验证集的 loss 曲线和 accuracy 曲线。



比较不同激活函数下 $train_loss$ | $train_acc$ | val_loss | val_acc | $time(s)$ 等数据的区别，简要分析哪组效果最好，可能原因是什么。

Train Loss 和 Train Acc

Relu 和 leaky_relu 表现相近且最优异，收敛速度快，损失最低，准确率最高

Tanh 表现不如 Relu 但比 sigmoid 强。

Val Loss 和 Val Acc

与 Train Loss 和 Train Acc 表现一样，relu 和 leaky_relu 表现最好，tanh 次之，sigmoid 表现最差

Time:

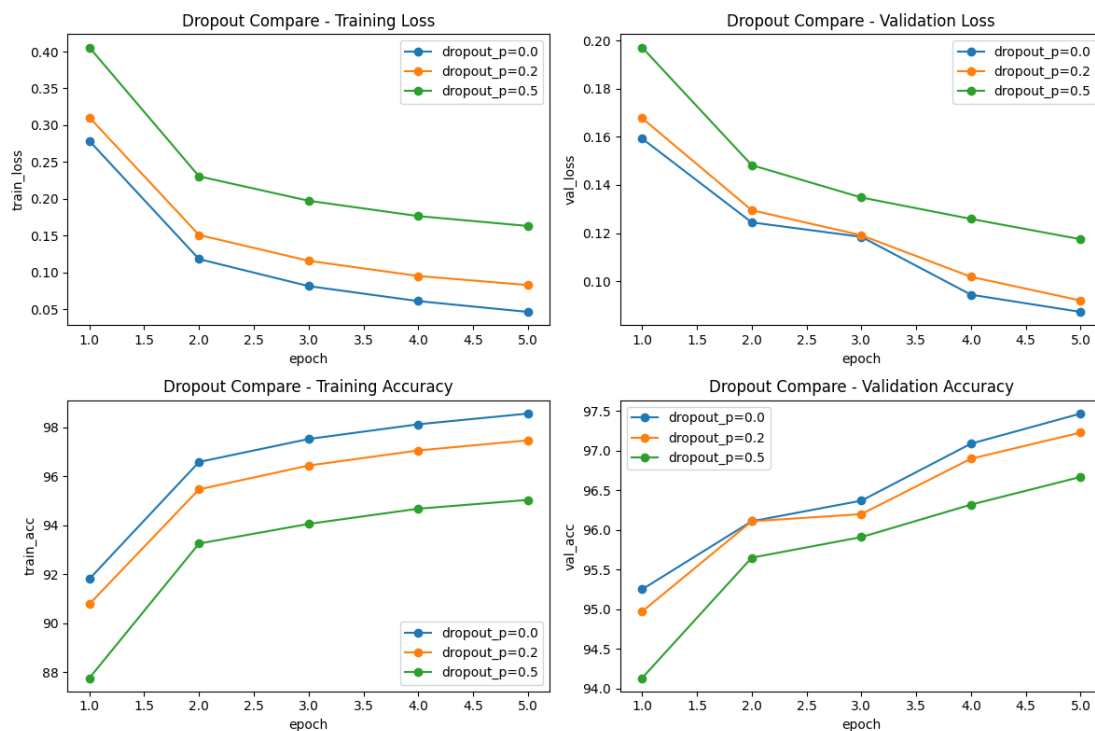
Relu 和 leaky_relu 最快，sigmoid 和 tanh 因为涉及到浮点运算会更慢

可能原因是：Relu 函数可以缓解梯度消失，同时计算更加高效，Sigmoid 函数在输入值绝对值较大的时候，导数趋近于 0，方向传播时梯度小，反向传播权重

无法更新，收敛缓慢。

(5) 添加 Dropout (过拟合观察)

设置不同 Dropout, 训练评估模型。截图训练集和验证集的 loss 曲线和 accuracy 曲线。



比较不同 Dropout 下 `train_loss` | `train_acc` | `val_loss` | `val_acc` | `time(s)` 等数据的区别，简要分析哪组效果最好，可能原因是什么。

Train Loss 和 Train Acc

Dropout 概率越大，模型在训练集的拟合能力越弱

Val Loss 和 Val Acc

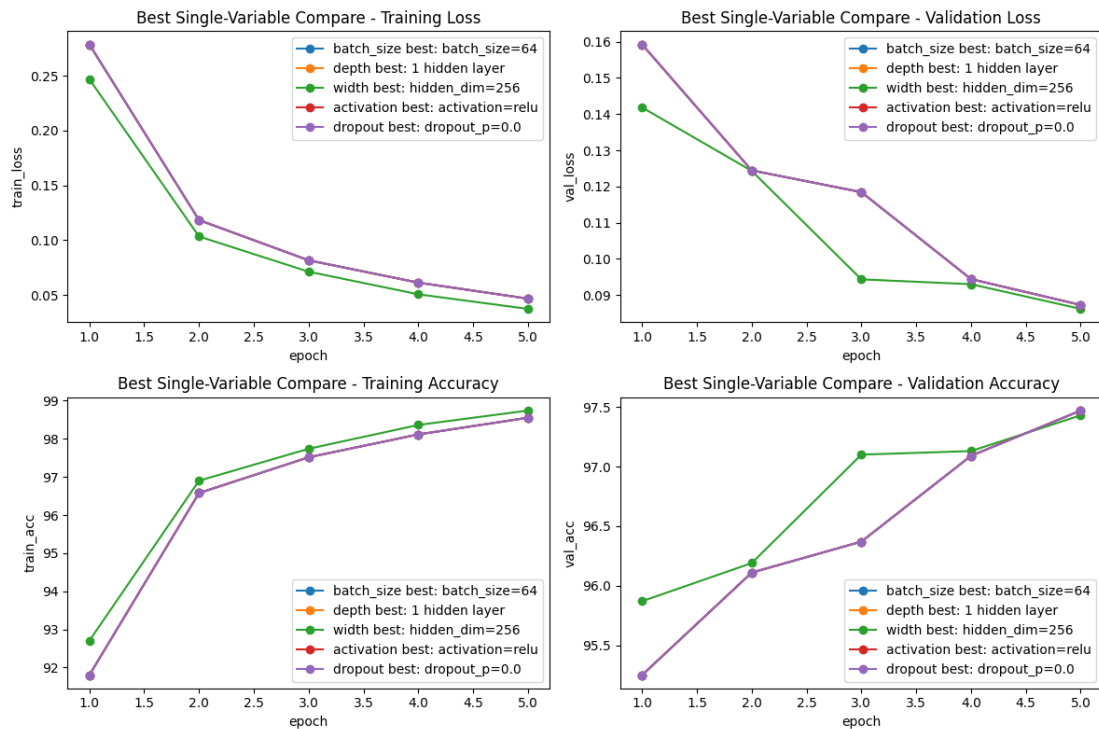
验证集和训练集的趋势一致，`dropout_p` 越小，验证损失越小，验证准确率越高

Time `dropoout_p=0` 小于 `dropout_p=0.2` 和 `dropout_p=0.5`

Dropout_p=0 效果最好，可能是因为模型尚未到达过拟合状态同时过高的 dropout_p 导致了过拟合趋势

(6) 不同调参方向的最佳单变量结果对比

请从每一类单变量实验中，依据验证集结果选出该类表现最好的设置，截图训练集和验证集的 loss 曲线和 accuracy 曲线。



Batch Size: 64

网络深度: 1 hidden layer

激活函数: relu

Dropout 概率: 0.0

隐藏层维度 : hidden_dim=256

回答下面的问题:

- 不同调参方向中，哪一类改动对模型性能影响最明显？
- 收敛速度更快的模型，最终验证效果是否一定更好？

c) 训练集准确率更高，是否一定代表模型泛化能力更强？

注意：回答时必须结合自己的实验数据和曲线，不要只写概念性结论。

a) 网络深度 和 激活函数 的改动对模型性能影响最明显。

例如 Relu 换成 Sigmoid，就会导致严重的梯度消失。网络深度 hidden_dim=512 导致在第四轮训练中出现过拟合。

b)：不一定，盲目追求极快的初始收敛速度，往往会牺牲模型在未知数据上的稳定性。

例如在 Batch_size 的训练上，batch_size=32 的组别在第 1 轮到第 2 轮时的训练损失下降最快，但是最终效果并不好。

c) 不是，训练集准确率过高，而验证集表现很差，是典型的过拟合现象，代表泛化能力极差。

(7) 测试集结果

在完成调参与对比实验后，选择较优参数配置，在测试集上进行一次最终评估，并记录测试结果。

```
实验名称: final_model(width: hidden_dim=512)
batch_size: 64
hidden_dims: [512]
activation_name: relu
dropout_p: 0.0
learning_rate: 0.001
epochs: 5
=====
Epoch 01/5 | train_loss=0.2209, train_acc=93.39% | val_loss=0.1256, val_acc=96.23% | time=4.25s
Epoch 02/5 | train_loss=0.0878, train_acc=97.29% | val_loss=0.1131, val_acc=96.72% | time=8.37s
Epoch 03/5 | train_loss=0.0580, train_acc=98.10% | val_loss=0.0855, val_acc=97.29% | time=6.97s
Epoch 04/5 | train_loss=0.0410, train_acc=98.68% | val_loss=0.1093, val_acc=96.66% | time=3.48s
Epoch 05/5 | train_loss=0.0338, train_acc=98.87% | val_loss=0.0903, val_acc=97.57% | time=3.40s
训练总耗时: 26.46 秒

最终模型最后 3 个 epoch 结果:
epoch | train_loss | train_acc | val_loss | val_acc | time(s)
-----|-----|-----|-----|-----|-----
3 | 0.0580 | 98.10% | 0.0855 | 97.29% | 6.97
4 | 0.0410 | 98.68% | 0.1093 | 96.66% | 3.48
5 | 0.0338 | 98.87% | 0.0903 | 97.57% | 3.40
Final test loss: 0.0832
Final test accuracy: 97.75%
```

Final test loss: 0.0832

Final test accuracy: 97.76%

四、 附加题（选做）

若完成附加题，请在此处按题目分别说明实现思路、关键代码片段、实验结果与分析。

（1）联合调参探索

在单变量实验基础上，同时调整多个参数，并与基线模型或最佳单变量结果进行对比分析。

（2）数据增强

仅对训练集使用 `RandomRotation`、`RandomCrop` 等数据增强方法，观察验证集或测试集结果的变化。

（3）自定义网络结构

设计不同于基础全连接 BP 网络的模型，如加入 `BatchNorm`、`Conv2d`、`MaxPool2d`、残差连接等，并与基线模型进行对比。

五、 实验总结与心得

（总结本次实验的收获，记录遇到的问题及解决方法。）

这次实验让我学会了如何调整参数，训练出优秀的模型，例如什么样的 `batch_size` 和激活函数是比较好的，让模型不会出现过拟合且有一定的泛化能力。